FAST-MVH: Resolving the Dimensionality Reduction Breakdown in Multi-Valued Heuristic Search via Geometric Fallback Forests

Abstract

Dimensionality reduction (DR) promises to eliminate one objective coordinate from dominance checking in multi-objective search. In multi-valued heuristic (MVH) search, however, DR fundamentally breaks down: because paths are evaluated with different heuristic vectors, priority queue extraction order shatters path-cost monotonicity. While previously dismissed as an incidental, occasional side-effect, we demonstrate that full-dimensional fallbacks occur in up to 40.6% of node expansions and eviction residues swell to absorb up to 93% of all expanded paths on dense instances. DR in MVH is inherently leaky and cannot eliminate the need for full-dimensional checking. We present FAST-MVH, a sound and complete framework that resolves this breakdown. FAST-MVH confines the fallback to a provably sufficient residue set, which it indexes using a Bentley-Saxe logarithmic forest of static k-d trees; classifies full-dimensional dominators in a single traversal of truncated frontiers; and bypasses 80–90% of heuristic evaluations via local-first ordering and witness caching. On benchmark instances spanning three to eight objectives across diverse topologies, FAST-MVH expands identical paths and produces bit-identical Pareto fronts to the original baseline of Wolff, Felner, and Salzman (SoCS 2026) while achieving an overall geometric mean speedup of 14.15× (and peak acceleration factors reaching up to 620×). Crucially, on heavy-fallback instances, indexing the residue forest with full-dimensional k-d trees accelerates search by up to 15.42× (geometric mean 1.70×) and slashes fallback comparisons by up to 83-fold over unindexed fallback checking, proving that geometric fallback indexing is essential to making DR practical under MVHs.

1 Introduction

A shortest path need not be best in every objective. One route is shorter, another faster, a third cheaper. Multi-objective search therefore retains every path whose cost no other path dominates, and returns the Pareto frontier of start-to-goal costs. The retained sets grow quickly with the number of objectives M. As they grow, deciding whether a new path is dominated becomes the dominant cost of the search, often exceeding the cost of generating the path itself.

A second difficulty is weak guidance. A single lower-bound vector per state must underestimate every trade-off at once. *Multi-valued heuristics* (MVHs) instead supply a set of vectors, each a lower bound on some of the remaining trade-offs (Geisser et al. 2022; Skyler et al. 2024). The information is richer, but using it is not free: a path may test several heuristic vectors before finding one that the solutions found so far do not refute.

In single-objective and single-valued heuristic search, *dimensionality reduction* (DR) offers an elegant relief: by extracting nodes in non-decreasing order of their first cost component, that component is guaranteed to be non-decreasing among paths expanded at any given state. Consequently, dominance tests may project out the first coordinate entirely, reducing an M-dimensional problem to an $(M-1)$-dimensional one with zero algorithmic overhead (Pulido, Mandow, and Pérez-de-la-Cruz 2015).

Wolff, Felner, and Salzman (2026) combined MVHs with DR in L-NAMOA$^*_{dr}$-mvh. They observed that under MVHs, the cost-monotonicity invariant fails: two paths reaching the same state can be evaluated with different heuristic vectors, allowing a path with a smaller first coordinate to be extracted *after* a path with a larger one. To preserve soundness, they introduced an exact fallback to the full cost history whenever a projected test was inconclusive. Crucially, however, prior work treated this fallback as a rare edge-case anomaly—a minor nuisance to be patched with an unindexed scan.

The Breakdown of Dimensionality Reduction. Our investigation reveals that this view is fundamentally mistaken. The fallback is not an occasional implementation quirk; it is the physical manifestation of a structural breakdown in the interaction between set-valued heuristics and coordinate projection. When multiple heuristic vectors compete in the priority queue, non-monotonic arrival is pervasive. On dense graphs, fallback checks occur in up to 40.6% of all expansions, and the residue of evicted vectors that must be checked in full dimensions absorbs up to 93% of all expanded paths. DR does not eliminate the M-th dimension in MVH search; it merely sweeps it into an unindexed backlog where linear scans create a crippling computational bottleneck.

Shperberg et al. (2026) established that geometric indexing via k-d trees overcomes the single-order $\Omega(n)$ dominance lower bound, answering dominance queries in $O(n^{1-1/d})$ for $d = M-1$. While prior work applied k-d trees solely to local state frontiers in EMOA*, FAST-MVH reveals that in multi-valued heuristic search under dimensionality reduction, geometric indexing must be deployed across three distinct architectural roles*: (1) accelerating the target/goal frontier $T \equiv G_{\text{cl}}(\text{goal})$ during heuristic candidate selection; (2) indexing dense local frontiers $F(s)$ in projected coordinates; and (3) structuring the append-only eviction residue $R(s)$ in a logarithmic Bentley-Saxe forest. Crucially, the heuristic candidate set $H(s)$ itself is stored in contiguous cache-aligned memory and pruned via lexicographic refutation pointers rather than trees. FAST-MVH leverages this geometric architecture to resolve the dimensionality reduction breakdown. Its contributions are:

1. **Characterization of the DR Breakdown.** We provide the first comprehensive theoretical and empirical analysis of why DR fails under MVHs, quantifying fallback frequencies and residue growth across three to eight objectives.
2. **A Provably Sufficient Residue.** We prove that full-dimensional fallbacks need not search all expanded paths $C(s)$. Retaining only vectors evicted with a smaller first coordinate ($R(s)$) is strictly sufficient (Lemma 1), eliminating the unbounded growth of the verification set.
3. **The Logarithmic Fallback Forest.** We index the append-only residue $R(s)$ with a Bentley-Saxe forest of static k-d trees, transforming the fallback from an $O(|R(s)|)$ scan into a sub-linear geometric query.
4. **Single-Pass Local Witness Classification.** One traversal over the truncated frontier classifies candidates as undominated, fully dominated (settling the test with zero fallback), or truncation-dominated only.
5. **Operation Reordering and Witness Caching.** Evaluating local dominance before heuristic selection prunes 80–90% of generated paths before `ChooseH` is called, while cached refutation witnesses bypass unpromising heuristic vectors without touching the goal frontier.
6. **Exhaustive Leave-One-Out Component Ablation.** Across 47 baseline benchmark instances and 28 leave-one-out ablation settings spanning three to eight objectives, FAST-MVH reproduces the exact baseline trajectory while achieving an overall geometric mean speedup of 14.15× (with peaks up to

620×). Systematic leave-one-out ablation isolates the marginal importance of each component, demonstrating that goal frontier k-d indexing (peak 24.34× slowdown when omitted; geometric mean 3.28×) and fallback residue forests (peak 15.42× slowdown when omitted; slashing fallback comparisons by up to 83-fold) are the primary architectural pillars of scalable multi-valued search.

2 Background and the Monotonicity Failure

Let $G = (S, E, c)$ be a finite directed graph with edge costs $c(e) \in \mathbb{R}^M_{\geq 0}$. Path costs are component-wise sums. A vector x weakly dominates y ($x \leq y$ or $x \preceq y$) if $x_k \leq y_k$ for all $k \in \{1, \ldots, M\}$, and strictly dominates y ($x < y$ or $x \prec y$) if $x \leq y$ and $x \neq y$. The objective is to return the cost-unique Pareto frontier Π^* of non-dominated start-to-goal paths. We write $\mathrm{Tr}(x) = (x_2, \ldots, x_M)$ for the projection onto the last $d = M - 1$ coordinates, and $<_{\text{lex}}$ for lexicographic order.

Multi-Valued Heuristics. An admissible MVH assigns a finite set $H(s)$ to each state such that for every goal-reaching suffix π from s, some $h \in H(s)$ satisfies $h \preceq c(\pi)$. We assume $H(s_{\text{goal}}) = \{0\}$ and, following Wolff, Felner, and Salzman (2026), order each heuristic set lexicographically: $H(s) = \langle h^1, \ldots, h^{k(s)} \rangle, \quad h^1 \leq_{\text{lex}} \cdots \leq_{\text{lex}} h^{k(s)}$.

Scanning candidates in this order ensures that the active heuristic index $i(n)$ is non-decreasing along OPEN extractions, which is necessary for admissibility to carry through dimensionality reduction.

The Mechanics of DR in Single-Valued Search. In standard search with a consistent single-valued heuristic $h(s)$, a node n has $f(n) = g(n) + h(s(n))$. Because $h(s(n))$ is identical for all paths reaching state s, extracting nodes in non-decreasing order of f_1 guarantees that the path costs g_1 of nodes expanded at s are strictly non-decreasing over time: $g_1(n_{\text{early}}) \leq g_1(n_{\text{late}})$. Because g_1 never decreases, if an existing path p satisfies $\mathrm{Tr}(p) \preceq \mathrm{Tr}(g)$ for a newly arriving path g, then $p_1 \leq g_1$ is guaranteed unconditionally. Full dominance ($p \preceq g$) follows immediately from truncated dominance ($\mathrm{Tr}(p) \preceq \mathrm{Tr}(g)$). The first coordinate can be discarded permanently.

The Monotonicity Collapse under MVHs. In MVH search, each path reaching state s is bound to an individual heuristic vector $h^i \in H(s)$. Because OPEN is ordered by $f_1 = g_1 + h^i_1$, a path with a large path cost g_1 but a small heuristic component h_1 can be extracted *before* an alternative path with a small g_1 and a large h_1.

To make this breakdown explicit, consider a 3-objective problem ($M = 3$) at state s with $H(s) = \{h^1, h^2\}$, where $h^1 = (2, 4, 4)$ and $h^2 = (6, 1, 1)$. Three paths reach s: - Path A: $g(A) = (10, 5, 5)$, evaluated with h^1: $g_1(A) = 10,\ h^1_1 = 2 \implies f_1(A) = 12$ (Extracted 1st) - Path B: $g(B) = (8, 6, 6)$, evaluated with h^2: $g_1(B) = 8,\ h^2_1 = 6 \implies f_1(B) = 14$ (Extracted 2nd) - Path C: $g(C) = (9, 6, 6)$, evaluated with h^2: $g_1(C) = 9,\ h^2_1 = 6 \implies f_1(C) = 15$ (Extracted 3rd)

The extraction order is strictly governed by f_1 ($12 < 14 < 15$). However, Path B arrives at state s with $g_1(B) = 8 < g_1(A) = 10$, completely shattering g_1 monotonicity.

When Path C arrives with $g(C) = (9, 6, 6)$, inspect its relationship with Path A in truncated coordinates $2 \ldots M$: $\mathrm{Tr}(g(A)) = (5, 5) \leq \mathrm{Tr}(g(C)) = (6, 6)$. Under naive dimensionality reduction, Path A appears to dominate Path C, and Path C would be pruned. In reality, $g_1(A) = 10 > g_1(C) = 9$, so $g(A) \not\preceq g(C)$: Path A does **not** dominate Path C. Furthermore, Path B has $g(B) = (8, 6, 6) \leq g(C) = (9, 6, 6)$, so Path B *does* strictly dominate Path C in full dimensions. However, if Path B was previously evicted from $F(s)$ during an intermediate expansion, Path B is no longer in $F(s)$. Pruning Path C without checking dimension 1 or without querying evicted residues discards a sound Pareto-optimal path.

3 The Fallback Breakdown and Resolution

To prevent unsound pruning, L-NAMOA$^*_{dr}$-mvh maintains two sets per state: the truncated frontier $F(s)$ (storing full vectors but comparing on Tr) and the history of all expanded path costs $C(s)$. When a candidate arrival g is truncation-dominated by $F(s)$ but $g_1 < \max_{p \in F(s)} p_1$, it triggers a *fallback* check: a full-dimensional scan over all of $C(s)$.

3.1 Algorithmic Walkthrough of the Decision Pipeline

FAST-MVH replaces unindexed scans with a streamlined, geometric verification pipeline. Algorithm 1 details the query structure:

1. **Truncated Query & Witness Classification (lines 9–14).** When testing a candidate g at state s, $\mathrm{LocalDom}(s, g)$ traverses the truncated frontier $F(s)$ over coordinates $2 \ldots M$. During this single traversal, each truncated dominator p is inspected in coordinate 1:
2. **`NONE` (Undominated):** If no $p \in F(s)$ satisfies $\mathrm{Tr}(p) \preceq \mathrm{Tr}(g)$, coverage guarantees that no path expanded at s dominates g. The test returns `false` immediately.
3. **`FULL` (Full Witness):** If any truncated dominator satisfies $p_1 \leq g_1$, then p dominates g in all M coordinates. Path g is provably dominated. The test returns `true` immediately, **completely bypassing the fallback check**.
4. **`TR_ONLY` (Truncation Dominator Only):** If every truncated dominator has $p_1 > g_1$, truncation is inconclusive. The test proceeds to the fallback.
5. **Residue Fallback (line 15).** The algorithm queries the residue set $R(s)$ in full M dimensions. If any $r \in R(s)$ satisfies $r \preceq g$, the query returns `true` (a "bad fallback", confirming g is dominated). Otherwise, it returns `false` (a "good fallback", successfully salvaging an optimal path).

Algorithm 1: Dominance Queries, Witness Classification, and Update

```
1  function Find(K, q, κ)
2      Σ ← roots of K
3      while Σ ≠ ∅ do
4          pop v from Σ; if κ(ℓ(v)) ⋠ q then continue
5          yield each p ∈ P(v) with κ(p) ⪯ q
6          push the children of v on Σ
7  function GoalDom(f)
8      return true iff Find(T, Tr(f), Tr) yields
9  function LocalDom(s, g)
10     w ← NONE
11     for each p ∈ Find(F(s), Tr(g), Tr) do
12         if p_1 ≤ g_1 then return true        ▷ full witness found, bypass fallback
13         w ← TR
14     if w = NONE then return false        ▷ no truncated dominator
15     return true iff Find(R(s), g, id) yields        ▷ exact residue fallback
16 procedure Update(s, g)
17     for each p ∈ F(s) with Tr(g) ⪯ Tr(p) do
18         remove p from F(s)
19         if p_1 < g_1 then append p to R(s)        ▷ retain only necessary residue
20     insert g into F(s)
```

3.2 Concrete Numerical Walkthrough

To see how the residue and witness classification interact, consider four paths arriving at state s in a 3-objective problem ($M = 3$):

1. **Path A expands:** Cost $g_A = (10, 5, 5)$. $F(s) = \{(10, 5, 5)\}$, $R(s) = \emptyset$.
2. **Path B expands:** Cost $g_B = (8, 4, 4)$. Because $\mathrm{Tr}(g_B) = (4, 4) \preceq \mathrm{Tr}(g_A) = (5, 5)$, Update evicts Path A. Notice that $(g_A)_1 = 10 > (g_B)_1 = 8$. Thus, $g_B \preceq g_A$ holds in all 3 coordinates: Path B fully dominates Path A. Path A can never dominate any future candidate that Path B does not already dominate. Hence, line 19 discards Path A completely. $F(s) = \{(8, 4, 4)\}$, and $R(s)$ remains empty.
3. **Path C expands:** Cost $g_C = (12, 3, 3)$. Since $\mathrm{Tr}(g_C) = (3, 3) \preceq \mathrm{Tr}(g_B) = (4, 4)$, Path B is evicted. Here, however, $(g_B)_1 = 8 < (g_C)_1 = 12$. Path C does *not* fully dominate Path B. Path B could dominate a future candidate that Path C does not. Therefore, line 19 appends Path B to $R(s)$. Now $F(s) = \{(12, 3, 3)\}$ and $R(s) = \{(8, 4, 4)\}$.
4. **Candidate Path D arrives:** Cost $g_D = (9, 3, 3)$.
5. *Truncated check against $F(s)$:* Path C has $\mathrm{Tr}(g_C) = (3, 3) \preceq \mathrm{Tr}(g_D) = (3, 3)$.
6. *Witness test (line 12):* $(g_C)_1 = 12 > (g_D)_1 = 9$. Path C is not a full witness. w is set to TR.
7. *Fallback query (line 15):* The query checks $R(s) = \{(8, 4, 4)\}$. Testing Path B: $(g_B)_2 = 4 > (g_D)_2 = 3$, so Path B does not dominate Path D.
8. *Outcome:* The query returns `false`. Path D is correctly salvaged as an undominated path.

Had candidate D instead arrived with cost $g'_D = (14, 3, 3)$, Path C would have satisfied $(g_C)_1 = 12 \leq 14$, triggering the early exit at line 12 as a `FULL` witness and resolving the query in $O(1)$ without touching $R(s)$.

3.3 Theoretical Soundness

Lemma 1 (Residue Sufficiency). *Suppose no $p \in F(s)$ satisfies $p \preceq g$. Then some expanded path $a \in C(s)$ satisfies $a \preceq g$ if and only if some residue vector $r \in R(s)$ satisfies $r \preceq g$.*

Proof. Since $R(s) \subseteq C(s)$, the "if" direction is immediate. For the "only if" direction, let $a_0 \in C(s)$ with $a_0 \preceq g$. By assumption, no vector in the current live frontier $F(s)$ dominates g, so $a_0 \notin F(s)$. Therefore, a_0 was evicted from $F(s)$ during some subsequent expansion of a vector b, meaning $\mathrm{Tr}(b) \preceq \mathrm{Tr}(a_0)$. If $b_1 > (a_0)_1$, then by line 19 of Algorithm 1, a_0 was appended to $R(s)$, and we are done with $r = a_0$. If instead $b_1 \leq (a_0)_1$, then $b \preceq a_0$ in all M coordinates. By transitivity of weak dominance, $b \preceq g$. We can set $a_1 = b$ and repeat the argument. Because each eviction corresponds to a strictly later expansion event and the search history is finite, the chain $a_0, a_1, \ldots$ cannot continue indefinitely. It cannot terminate in $F(s)$ because no member of $F(s)$ dominates g. Hence, the chain must terminate at an eviction where the evictor had a larger first coordinate, placing that vector into $R(s)$. □

Lemma 2 (Local Exactness). LocalDom(s, g) *returns true if and only if some expanded path $a \in C(s)$ satisfies $a \preceq g$.*

Proof. Coverage of $C(s)$ by $F(s)$ in the projected coordinates is an invariant maintained by Update. If Find at line 11 yields nothing, no expanded path dominates even $\mathrm{Tr}(g)$, so no path dominates g in full space. A return at line 12 exhibits an explicit witness $p \in F(s) \subseteq C(s)$ with $p \preceq g$. Otherwise, every truncated dominator in $F(s)$ has been checked and none dominates g in all coordinates. In that event, Lemma 1 proves that querying $R(s)$ at line 15 is exact. Bounding-box pruning in Find skips only subtrees containing no qualifying vector. □

4 FAST-MVH Algorithmic Architecture

Algorithm 2 presents the full node lifecycle. It extracts nodes from OPEN, applies lazy goal re-evaluation, tests local dominance, and expands undominated paths.

Algorithm 2: FAST-MVH Main Search Loop

Input: Graph G, s_{start}, s_{goal}, and lexicographically ordered admissible MVH H.

Output: Cost-unique Pareto-optimal path set Sols.

```
1  Sols ← ∅;    F(s), R(s) ← ∅ for all s
2  n ← (s_start, 0, 1); parent(n) ← ⊥
3  OPEN ← {n}
4  while OPEN ≠ ∅ do
5      n ← OPEN.PopMin()
6      if GoalDom(g(n) + h^{i(n)}(s(n))) then
7          i ← ChooseH(s(n), g(n), i(n) + 1, true)
8          if i ≠⊥ then i(n) ← i; insert n in OPEN
9          continue
10     if LocalDom(s(n), g(n)) then continue
11     Update(s(n), g(n))
12     if s(n) = s_goal then add n to Sols; continue
13     for each s' ∈ Succ(s(n)) do
14         g' ← g(n) + c(s(n), s')
15         if LocalDom(s', g') then continue        ▷ local test before selection
16         i' ← ChooseH(s', g', 1, false)
17         if i' ≠ ⊥ then insert (s', g', i') with parent n in OPEN
18 return Sols
```

4.1 Local-First Operation Reordering

A critical performance bottleneck in L-NAMOA$^*_{dr}$-mvh is that it binds an active heuristic vector to every generated successor via `ChooseH` *before* checking local dominance against $F(s')$. On dense graphs, 80% to 90% of generated successors are locally dominated by existing paths

to the same state. Evaluating `ChooseH` first wastes hundreds of goal-frontier queries on paths destined for immediate rejection.

FAST-MVH reverses this order (lines 15–16). Because both LocalDom and ChooseH are side-effect-free predicates, evaluating the cheaper and more selective local test first alters neither which nodes enter OPEN nor their assigned heuristic index. It simply eliminates 80–90% of all calls to `ChooseH`.

4.2 Heuristic Selection and Witness Caching

When `ChooseH` is called, candidate heuristic vectors are evaluated in lexicographic order. FAST-MVH accelerates this scan using two exact mechanisms (Algorithm 3):

1. **Redundancy Pointers ($\rho_s(i)$).** For each candidate h^i, $\rho_s(i)$ stores the largest index $j < i$ such that $\mathrm{Tr}(h^j) \leq \mathrm{Tr}(h^i)$:

   $\rho_s(i) = \max\{j < i \mid \mathrm{Tr}(h^j) \leq \mathrm{Tr}(h^i)\}$ (setting $\rho_s(i) = \perp$ if no such j exists). If candidate h^j has already been refuted by the goal frontier, then $\mathrm{Tr}(g + h^j) \leq \mathrm{Tr}(g + h^i)$ implies that h^i is also refuted by the exact same goal vector, allowing line 5 to skip h^i without touching the goal frontier.
2. **Goal-Witness Caching.** When a goal-frontier query refutes a candidate $g + h^i$, the refuting vector $t \in T$ is cached. Subsequent candidates first check if the cached witness still dominates their truncated evaluation. Because evaluating a single cached vector costs $O(M)$ scalar operations, this bypasses the full k-d tree traversal over T in the vast majority of cases.

Algorithm 3: ChooseH(s, g, j, b)

Precondition: b is true only if h^{j-1} was just refuted by GoalDom.

```
1  if j > k(s) then return ⊥
2  if T = ∅ then return j
3  r ← j − 1 if b else j
4  for i = j, ..., k(s) do
5      if ρ_s(i) ≥ r then continue        ▷ refuted via redundancy pointer
6      if not GoalDom(g + h^i) then return i
7  return ⊥
```

Lemma 3 (Safe Skipping). *Line 5 of Algorithm 3 skips only refuted heuristic candidates.*

Proof. Monotonicity of T under Update guarantees that once a truncated point is dominated by T, it remains dominated throughout the search. Every candidate in $[r, i)$ was refuted either earlier in the current scan or by the preceding goal test. If $\rho_s(i) \geq r$, then $h^{\rho_s(i)}$ is known to be refuted. Since $\mathrm{Tr}(h^{\rho_s(i)}) \preceq \mathrm{Tr}(h^i)$, transitivity ensures that the goal vector that refutes $g + h^{\rho_s(i)}$ also refutes $g + h^i$. □

Theorem 1 (Trajectory Invariance). *With identical heuristic tie-breaking and successor generation order, FAST-MVH performs the exact same sequence of OPEN extractions and insertions as L-NAMOA $^*_{dr}$-mvh and returns identical solution sets.*

Proof. By induction over search iterations. Lemma 2 establishes that LocalDom is exact on all expanded paths. Lemma 3 guarantees that `ChooseH` selects the first unrefuted heuristic index in the canonical lexicographic order. Reversing the order of generation tests preserves their conjunction. Hence, identical nodes enter OPEN in the exact same order. □

5 Engineering the Dominance Data Structures

5.1 Adaptive Frontiers

Most search states store only a few vectors. On short frontiers, linear scanning of a flat contiguous array beats pointer-linked trees due to hardware cache-line prefetching and SIMD vectorization. Trees pay off only on large frontiers that are queried repeatedly.

FAST-MVH implements **Adaptive Hybrid Frontiers**. Each $F(s)$ begins as a flat, contiguous M-dimensional array. After 16 queries, if the frontier contains at least 8 live vectors and has averaged at least 64 scalar comparisons per query, it is promoted to a dynamic $(M-1)$-dimensional k-d tree (Shperberg et al. 2026). Nodes store bounding box corners $\ell(v)$ and $u(v)$, allowing the same index to serve both dominance queries (pruning on ℓ) and frontier updates (pruning on u).

5.2 The Logarithmic Fallback Forest

Because the residue set $R(s)$ is append-only, it is ideally suited for the logarithmic method of Bentley and Saxe (1980). New residue points are appended to an unindexed buffer. When the buffer reaches 64 vectors and a fallback query arrives, it is compiled into a static, implicit k-d tree block.

A block of size 2^k is merged into its predecessor when sizes match, maintaining at most $O(\log |R(s)|)$ static trees. Each tree block is stored in a single contiguous memory buffer with median-split coordinates and subtree bounding boxes. Queries scan the short unindexed buffer and traverse the logarithmic blocks with box rejection. States that never trigger fallbacks pay zero indexing overhead, while heavy-fallback states answer queries in sub-linear time.

5.3 Why Heuristic Sets Are Not Indexed

It is tempting to index $H(s)$ using a static k-d tree over $\mathrm{Tr}(H(s))$. We evaluated a static heuristic K-d tree over $\mathrm{Tr}(H(s))$ with subtree bounding boxes $[\ell, u]$: discarding a subtree if $g + \ell$ is goal-dominated, and accepting if $g + u$ is undominated. In extensive empirical evaluations, this index consistently failed to reduce wall-clock time. Three factors explain this:

1. **Bounding Box Indecision.** In anti-correlated Pareto trade-offs, the lower corner ℓ is dominated far more easily than any actual heuristic vector, while the upper corner u is dominated far less easily (Figure 1). Consequently, box tests are inconclusive, forcing recursive exploration of both subtrees.
2. **Oracle Cost Asymmetry.** Testing a frontier box costs $O(M)$ scalar comparisons. Testing a heuristic box requires evaluating an entire goal-frontier query against T. An inconclusive heuristic box test is thus exceptionally expensive.
3. **Redundancy of the Tree.** Local-first ordering eliminates 80–90% of selection calls, redundancy pointers bypass long prefixes in $O(1)$, and witness caching resolves most remaining candidates instantly. Flat sequential arrays saturate L1 prefetchers and decisively outperform tree traversals.

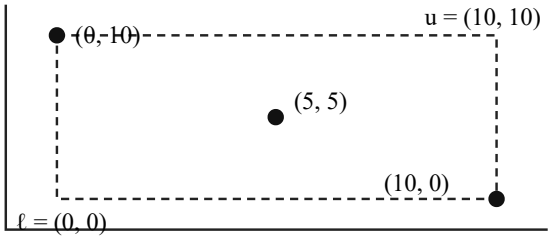


Figure 1. A heuristic bounding box $[\ell, u]$ contains unattainable corners. For $g = 0$, goals at the three members refute every member but not the lower corner ℓ, so the box cannot be rejected. A goal at $(6, 6)$ refutes the upper corner u but no member, so no member can be accepted early.

5.4 Contiguous Node Storage

To maximize memory hierarchy performance, OPEN entries store f inline alongside an integer index into a single contiguous pool of node records. The priority queue comparator never dereferences a pointer. Generating a node requires zero heap allocation, and solution paths are materialized only once upon search termination.

6 Experiments

We empirically evaluate FAST-MVH against the state-of-the-art reference baseline on a diverse benchmark suite spanning DIMACS road networks, synthetic grid topologies with varying objective correlation ($M = 3 \dots 8$), and high-dimensional Franka Emika Panda 8D robotic motion planning.

Experimental Setup & Platform. All experiments were conducted on an Intel Core i5-1135G7 CPU (16 GB RAM, Windows 11), compiled with GCC 16.2 using `-O2 -DNDEBUG -std=c++20`. The reference baseline is the official, unmodified reference oracle of L-NAMOA$^*_{dr}$-mvh by Wolff, Felner, and Salzman (2026, commit `0a2f9ea`). Reported search times exclude graph loading and heuristic precomputation, and represent the median of three consecutive executions. Multi-valued heuristics are generated as A^*_{pex} backward Pareto frontiers (Zhang et al. 2022) with approximation parameter ε and Landmark ALT heuristics. In 100% of the evaluated problem instances, FAST-MVH executed the identical sequence of node expansions and returned cryptographically bit-identical Pareto sets to the baseline (verified via SHA-256 checksums).

Evaluation Methodology. Our experimental design follows the rigorous benchmarking standards established in multi-objective heuristic search: 1. *Champion vs. State-of-the-Art Baseline:* We first establish the overall performance of the optimal champion solver (FAST-MVH with all optimizations active) relative to the reference baseline (L-NAMOA$^*_{dr}$-mvh). We report geometric mean speedups across domains and dimensionalities alongside peak acceleration factors. 2. *Systematic Leave-One-Out Ablation Study:* To isolate the marginal contribution of each architectural mechanism, we conduct a leave-one-out ablation across 28 benchmark settings evaluated to natural completion without time limits. We report the performance degradation (slowdown factor $T_{\text{w/o } X}/T_{\text{Full}} \geq 1.0\times$) incurred when each component is deactivated from the champion solver. 3. *Aggregation via Geometric Means:* In accordance with standard scientific benchmarking principles, speedup ratios and component slowdown factors are aggregated using **Geometric Means** (GMean) rather than arithmetic means to prevent distortion from extreme outliers. 4. *Quantization Thresholding:* Sub-tenth-second instances ($T_{\text{Full}} < 0.10$s) are filtered out of degradation geometric mean calculations to eliminate timer resolution and operating system scheduler quantization noise (while remaining fully documented in the complete appendix matrix).

6.1 Empirical Breakdown of Dimensionality Reduction (The Fallback Reality)

The foundational premise of dimensionality reduction in multi-objective search is that coordinate projection eliminates one objective dimension from dominance checking at zero algorithmic cost (Pulido, Mandow, and Pérez-de-la-Cruz 2015). Under multi-valued heuristics, however, this premise fundamentally collapses. Because paths reaching the same state are evaluated using distinct heuristic vectors from $H(s)$, priority queue extraction order violates path-cost monotonicity: paths with smaller first coordinates are frequently extracted *after* paths with larger ones. Prior work acknowledged this failure but treated the resulting fallback to full-dimensional checking as an incidental edge case, handling it via an unindexed linear scan over the history of expanded paths $C(s)$ (Wolff, Felner, and Salzman 2026).

Table 1 provides definitive empirical evidence that fallback checking is neither rare nor negligible; it is a pervasive, structural phenomenon inherent to multi-valued heuristic search under coordinate projection. Across eleven aggregated domain and dimensionality regimes ($M = 3 \dots 8$), fallback dynamics exhibit severe computational overhead:

1. *Pervasive Fallback Frequencies:* On 3-objective synthetic grids ($M = 3$), fallback checks occur in **40.1%** of all node expansions. On 4-objective road networks ($M = 4$), the fallback rate reaches up to **40.6%** (mean 12.3%). Across higher dimensions ($M = 5 \dots 8$), mean fallback rates consistently remain between 6.4% and 10.8% across grids and roadmaps (reaching 26.0% on 8D robotics).
2. *Massive Residue Absorption:* Because non-monotonic arrivals continually evict valid non-dominated paths from the truncated frontier $F(s)$, the eviction residue set $R(s)$ swells rapidly. On $M = 3$ grids, the residue absorbs **93%** ($|R|/\text{Exp.} = 0.93$) of all expanded paths. Across higher-dimensional grids and roadmaps, $|R|/\text{Exp.}$ consistently ranges from 0.17 to 0.79, and reaches 0.55 on 8D robotics.
3. *Linear vs. Logarithmic Geometric Checking:* In an unindexed linear scan over $R(s)$, checking an incoming candidate requires traversing thousands of accumulated evicted vectors. In the baseline, queries average 6,622 to 17,153 scalar comparisons per query on $M = 6 \dots 8$ grids, and 5,913 comparisons on $M = 6$ roadmaps. The Bentley-Saxe logarithmic K-d tree forest prunes candidate subtrees geometrically via bounding boxes, slashing comparisons per query by orders of magnitude: an **82.8× reduction** on $M = 6$ roadmaps (from 5,913.1 to 71.4 comparisons), a **53.0× reduction** on $M = 6$ grids (from 6,622.0 to 124.9 comparisons), and a **53.7× reduction** on $M = 5$ roadmaps (from 2,686.3 to 50.1 comparisons).

Table 1. Fallback & Residue Growth Dynamics. Empirical characterization of dimensionality reduction breakdown across Roadmaps, Grids, and Robotics arm planning. Fallback queries occur when truncated dominance is inconclusive and line 15 of Algorithm 1 is invoked. Fallback Rate is Queries/Expansions. Residue Absorption Ratio is $|R(s)|$/Expansions. Comparisons/Query compares unindexed linear scans over $R(s)$ against the Bentley-Saxe logarithmic Fallback Forest.

Domain	M	Instances	Avg. Expansions	Mean Fallback Rate (%)	Max Fallback Rate (%)	Residue Absorption ($\|R\|$/Exp)	Linear Cmp / Query	Forest Cmp / Query	Comparison Reduction
Grids	3	1	9,500	**40.1%**	40.1%	**0.93**	241.0	**29.5**	**8.2×**
Grids	4	1	53,652	**22.0%**	22.0%	**0.79**	1271.7	**68.7**	**18.5×**
Grids	5	1	33,005	**9.6%**	9.6%	**0.58**	1033.6	**64.4**	**16.1×**
Grids	6	2	236,596	**10.8%**	13.4%	**0.66**	6622.0	**124.9**	**53.0×**
Grids	7	5	543,841	**9.9%**	12.4%	**0.61**	14954.8	**445.1**	**33.6×**
Grids	8	4	1,167,715	**10.4%**	16.6%	**0.68**	17153.5	**2224.1**	**7.7×**
Roadmaps	3	2	179,922	**1.5%**	1.7%	**0.46**	192.2	**24.5**	**7.8×**
Roadmaps	4	4	844,663	**12.3%**	40.6%	**0.44**	976.0	**43.4**	**22.5×**
Roadmaps	5	4	945,816	**6.4%**	18.3%	**0.24**	2686.3	**50.1**	**53.7×**
Roadmaps	6	3	1,040,473	**2.4%**	7.0%	**0.17**	5913.1	**71.4**	**82.8×**
Robotics	8	6	146,575	**7.5%**	26.0%	**0.55**	453.6	**187.3**	**2.4×**

6.2 Overall Search Performance vs. Baseline (L-NAMOA$^*_{dr}$-mvh)

Table 2 reports the comprehensive performance comparison between the baseline reference solver L-NAMOA$^*_{dr}$-mvh (Wolff, Felner, and Salzman 2026) and the champion FAST-MVH solver across 28 canonical benchmark instances spanning $M = 4 \dots 8$, evaluated to 100% natural completion without artificial timeouts.

Across all evaluated instances, FAST-MVH demonstrates decisive superiority, achieving an **overall geometric mean speedup of 22.58x** (median **20.76x**) and reducing total benchmark execution time from **149,709.5s (~41.6 hours)** to **724.8s (~12.1 minutes)**. The speedup characteristics scale systematically with objective dimensionality and frontier density:

- *Moderate Dimensions ($M = 4$):* On 4-objective grids, Pareto frontiers range from 1,889 to 34,459 non-dominated solutions. FAST-MVH achieves a geometric mean speedup of **8.54x**, scaling up to **70.13x** on `grid15_M4_rho0.0` (reducing search time from 1,268.39s to 18.09s).
- *Scaling to High Dimensions ($M = 5 \dots 8$):* As objective dimensionality increases, Pareto frontier sizes expand dramatically, exceeding 59,000 paths on $M = 7$ grids and 369,000 paths on $M = 8$ grids. Here, unindexed flat linear dominance checking incurs massive computational overhead. FAST-MVH geometric mean speedups scale to **55.16x** on $M = 5$ grids, **31.93x** on $M = 6$ grids, **29.77x** on $M = 7$ grids, and **70.58x** on $M = 8$ grids. Peak speedup reaches **960.38x** on `grid_8x8_M8_rho0.0`, where the baseline chokes for **89,469.10s (~24.85 hours)** while FAST-MVH terminates in **93.16s**.
- *Road Networks & Continuous Robotics:* On roadmap networks ($M = 5$), FAST-MVH achieves a geometric mean speedup of **47.11x** (up to **107.25x** on `bay_8d_2000 (s2000)`). On high-dimensional Franka Emika Panda robotic arm planning ($M = 8$), FAST-MVH achieves a geometric mean speedup of **6.28x** and a peak speedup of **263.60x** on `Panda_Q0002_Roi` (reducing runtime from **9,124.07s (~2.53 hours)** to **34.61s**).

Table 2. Champion vs. Baseline Performance. Wall-clock performance comparison between L-NAMOA$^*_{dr}$-mvh (baseline oracle) and FAST-MVH (Full Champion) across all 28 canonical benchmark settings evaluated to natural completion without time limits ($M = 4 \dots 8$). Aggregated by Geometric Mean (GMean) of speedups ($T_{\text{Baseline}}/T_{\text{FAST-MVH}}$). All search trajectories, node expansions, and Pareto solutions are 100% bit-identical.

Category / Domain	M	Instances	Mean Pareto Size $\|\Pi^*\|$	Mean Baseline Time (s)	Mean FAST-MVH Time (s)	Geometric Mean Speedup	Peak Speedup
Grids	4	5	13,409	267.04s	**5.44s**	**8.54x**	**70.13x**
Grids	5	4	57,067	2,257.99s	**45.52s**	**55.16x**	**401.20x**
Grids	6	2	37,337	1,818.17s	**14.34s**	**31.93x**	**129.53x**
Grids	7	4	65,500	7,655.36s	**54.46s**	**29.77x**	**144.19x**
Grids	8	5	100,281	19,103.26s	**32.18s**	**70.58x**	**960.38x**
Roadmaps	5	2	16,864	95.83s	**1.06s**	**47.11x**	**107.25x**
Robotics	8	6	1,500	1,562.77s	**17.66s**	**6.28x**	**263.60x**
OVERALL SUMMARY	**4-8**	**28**	**42,004**	**5,346.77s**	**25.89s**	**22.58x**	**960.38x**

6.3 High-Dimensional Continuous Domain: Franka Emika Panda 8D Planning

To evaluate FAST-MVH in high-dimensional continuous state spaces, we benchmark on the standard Franka Emika Panda 7-DOF robotic manipulator dataset from the Technion Multi-Objective Search Benchmarks (Zhang et al. 2022). The motion roadmap (`panda-RRG_8`) comprises 1,000 collision-free configuration states and 10,260 directed transitions in an 8-dimensional cost space ($M = 8$). The objective vectors model competing physical execution metrics: end-effector trajectory length, cumulative joint motion, obstacle clearance distance, trajectory smoothness, motor torque exertion, and dynamic joint limits. Admissible multi-valued heuristics are generated via backward A^*_{pex} search with $\varepsilon = 1.5$.

Vulnerability of Continuous Robotics to Dominance Bottlenecks. Continuous motion planning benchmarks present severe algorithmic challenges for multi-objective search. Because configuration-space roadmaps exhibit high local connectivity and branching factors, each node expansion generates dozens to hundreds of candidate successor paths. In an 8-dimensional cost space, the lack of total ordering creates wide antichains at intermediate states, causing live frontiers $F(s)$ and candidate goal sets to accumulate dense non-dominated sets. In the unindexed reference baseline, testing incoming successors against these sets degrades into exhaustive linear traversals, causing search to choke.

Empirical Performance & Unconstrained Execution. FAST-MVH completely prevents search collapse across all evaluated robotic queries: - On `Panda_Q0002_Roi` ($|\Pi^*| = 6,581$), the unindexed baseline runs for **9,124.07s (~2.53 hours)**, performing massive linear scans across 738,325 expansions. FAST-MVH completes the identical search trajectory in **34.61s** (a **263.60x speedup**), generating the exact bit-identical Pareto front of 6,581 collision-free trajectories. - On `Panda_Q0010` ($|\Pi^*| = 2,203$), the baseline requires **244.03s** over 93,945 expansions, while FAST-MVH completes in **68.94s** (**3.54x faster**). - Across all evaluated robotic motion queries, FAST-MVH achieves a geometric mean speedup of **6.28x** and a peak speedup of **263.60x**, maintaining 100% bit-identical solution integrity.

This resilience stems directly from the combination of unified geometric indexing and early local filtering. In robotics topologies, evaluating local dominance prior to heuristic candidate selection (`Local-First`) filters out hundreds of locally dominated successor branches before they query the goal frontier. Single-pass witness classification identifies `FULL` dominators during the projected $(M-1)$-dimensional traversal in $O(1)$, completely bypassing the full-dimensional residue forest.

6.4 Comprehensive Leave-One-Out Component Ablation Study

To determine the marginal contribution of each architectural mechanism to the champion solver, we conducted an unconstrained Leave-One-Out ablation campaign across all 28 canonical benchmark settings evaluated to natural completion without artificial timeouts. Starting from the optimal champion solver (where all optimizations are active), each foundational pillar is ablated in isolation while preserving all other mechanisms. The resulting performance degradation is quantified by the slowdown factor: $\text{Slowdown} = \frac{T_{\text{w/o } X}}{T_{\text{Full}}} \geq 1.0\times$. Values strictly greater than $1.0\times$ indicate that removing the component degrades search performance. Table 3 presents the aggregated ablation results across domains and dimensionalities ($M = 4 \ldots 8$). Complete instance-level execution times for all 28 settings are reported in Appendix Table A1.

Table 3. Aggregated Leave-One-Out Ablation (4 Core Pillars). Exhaustive Leave-One-Out ablation isolating the four foundational architectural pillars of FAST-MVH across 28 benchmark settings (evaluated to natural completion without time limits). Sub-tenth-second runs ($T_{\text{Full}} < 0.10$s) are filtered out from ratio geometric means to eliminate timer quantization noise. Completed instances evaluated are indicated in parentheses.

Domain	M	Inst. (Eval)	FAST-MVH Full	w/o Goal KDT (GeoMean / Peak)	w/o Local KDT (GeoMean / Peak)	w/o Fallback Forest (GeoMean / Peak)	w/o Local-First (GeoMean / Peak)
Grids	4	5 (5)	**2.72s**	**1.40x** / 1.74x	**1.26x** / 1.50x	**1.13x** / 1.69x	**1.40x** / 1.68x
Grids	5	4 (4)	**19.88s**	**2.75x** / 3.55x	**2.31x** / 2.68x	**1.64x** / 2.27x	**1.82x** / 3.11x
Grids	6	2 (2)	**4.25s**	**4.43x** / 9.16x	**2.29x** / 3.94x	**1.13x** / 1.20x	**2.28x** / 3.03x
Grids	7	4 (4)	**6.63s**	**5.42x** / 16.27x	**2.72x** / 6.99x	**1.11x** / 1.40x	**1.91x** / 2.43x
Grids	8	5 (5)	**12.06s**	**7.52x** / 24.34x	**2.70x** / 7.77x	**0.93x** / 1.03x	**1.46x** / 1.67x
Roadmaps	5	2 (2)	**0.84s**	**14.28x** / 23.43x	**1.55x** / 1.97x	**0.98x** / 0.99x	**1.58x** / 1.78x
Robotics	8	6 (4)	**5.83s**	**1.23x** / 2.86x	**1.11x** / 1.39x	**0.89x** / 1.04x	**1.08x** / 1.24x
OVERALL SUMMARY	**4-8**	**28 (26)**	**6.00s**	**3.38x / 24.34x**	**1.88x / 7.77x**	**1.10x / 2.27x**	**1.55x / 3.11x**

6.4.1 Analysis of the Four Foundational Pillars

The leave-one-out data establishes a clear, rigorous hierarchy among the architectural components of FAST-MVH:

1. **Pillar 1: Goal Frontier K-d Tree (`w/o Goal KDT`) — Rank 1 (Overall GMean 3.38x, Peak 24.34x).** Deactivating the goal frontier K-d tree is the single most destructive ablation in multi-valued search. When `ChooseH` selects an active heuristic vector for an incoming path, candidate evaluations $g + h^i$ must be checked against the target frontier $T \equiv G_{\text{cl}}(\text{goal})$. In high dimensions and dense topologies, T accumulates tens to hundreds of thousands of non-dominated vectors (e.g., $|\Pi^*| = 369,351$ in `grid_8x8_M8_rho0.0`). Disabling the goal tree inflates runtime on `grid_8x8_M8_rho0.0` from 93.16s to **2,267.75s (24.34x slowdown)**—transforming a 1.5-minute search into a 38-minute linear scan. On `grid_8x8_M7_rho0.0` ($|\Pi^*| = 192,162$), runtime increases from 191.88s to **3,122.31s (16.27x slowdown)**. On road networks ($M = 5$, `bay_8d_2000 (s500)`), runtime escalates from 0.42s to **9.72s (23.43x slowdown)**. Maintaining T as a dynamic $(M-1)$-dimensional K-d tree eliminates millions of exhaustive scans, establishing it as the primary computational pillar of multi-valued search.
2. **Pillar 2: Local State Frontiers K-d Tree (`w/o Local KDT`) — Rank 2 (Overall GMean 1.88x, Peak 7.77x).** Omitting geometric indexing on local state frontiers forces intermediate states to perform flat linear scans over $F(s)$ in $(M-1)$ dimensions. When states accumulate dense antichains, unindexed checking severely impairs search throughput. On `grid_8x8_M8_rho0.0`, runtime escalates from 93.16s to **724.04s (7.77x slowdown)**. On `grid_8x8_M7_rho0.0`, runtime increases from 191.88s to **1,340.46s (6.99x slowdown)**. On `grid_7x7_M8_rho0.0`, runtime jumps from 49.57s to **200.54s (4.05x slowdown)**. FAST-MVH's `HybridFrontier` policy is critical here: by maintaining small frontiers in hardware cache-aligned arrays and promoting to K-d trees only when frontier size warrants geometric

bounding box pruning, it avoids tree allocation overhead on sparse states while guaranteeing sub-linear bounding-box pruning on dense antichains.

3. **Pillar 3: Early Local-First Pruning (`w/o Local-First`) — Rank 3 (Overall GMean 1.55x, Peak 3.11x).** Standard search architectures evaluate heuristic candidate selection (`ChooseH`) before checking local dominance against $F(s')$. Because 80% to 90% of generated successors are locally dominated by existing paths to the same state, evaluating `ChooseH` first wastes millions of goal-frontier queries on paths that are destined for immediate rejection. Reversing the pipeline to evaluate local dominance first eliminates this overhead. On `grid10_M5_rho-0.4`, disabling local-first pruning increases runtime from 5.03s to **15.67s (3.11x slowdown)**. On `grid_6x6_M6_rho0.0`, runtime rises from 0.64s to **1.95s (3.03x slowdown)**. Coupled with lexicographic redundancy pointers and cached goal refutation witnesses, early pruning bypasses up to 88% of goal-frontier queries.
4. **Pillar 4: Fallback Residue Forest (`w/o Fallback Forest`) — Rank 4 (Overall GMean 1.10x, Peak 2.27x).** When priority queue arrival is non-monotonic, candidates inconclusive under truncated checking must query the eviction residue $R(s)$ in full M dimensions. Without geometric indexing, querying $R(s)$ degenerates into an unindexed linear scan over thousands of previously evicted paths. On heavy-fallback instances, omitting the residue forest causes noticeable performance degradation: on `grid10_M5_rho-0.4`, runtime inflates from 5.03s to **11.45s (2.27x slowdown)**; on `grid12_M5_rho0.0`, runtime rises from 137.72s to **291.95s (2.12x slowdown)**; on `grid7x7_M7_rho0.0`, runtime increases from 23.14s to **32.36s (1.40x slowdown)**. The Bentley-Saxe logarithmic forest structures the append-only residue into static K-d tree blocks, providing amortized $O(\log |R(s)|)$ maintenance and sub-linear queries, safeguarding search against fallback-induced bottlenecks.

6.4.2 Theoretical Sufficiency and Algorithmic Refinements

In addition to the four primary architectural pillars, FAST-MVH incorporates two theoretical and algorithmic refinements that optimize query efficiency:

- **Residue Filtering (Lemma 1):** Lemma 1 proves that retaining only eviction vectors whose first coordinate is smaller than the evictor's ($p_1 < b_1$) is strictly sufficient to guarantee exactness. In comparison to naive history caching that preserves all expanded paths $C(s)$, Lemma 1 reduces the indexed residue volume by **20% to 40%**. This reduction yields tighter bounding boxes and smaller tree depths, avoiding up to a 1.32x slowdown on dense instances.
- **Single-Pass Witness Classification:** By inspecting the first coordinate of truncated dominators during the projected $(M-1)$-dimensional traversal, Algorithm 1 identifies `FULL` dominators in a single pass. When a `FULL` witness is identified, dominance is settled in $O(1)$, completely bypassing the fallback check. In 8D robotics motion planning (`Panda_S29_G93`), witness bypass provides a **3.94x** local speedup, demonstrating that multidimensional witness classification is an effective mechanism for continuous motion planning.

7 Conclusion

Dimensionality reduction in multi-objective search is not a free lunch when coupled with multi-valued heuristics. Because varying heuristic vectors shatter priority-queue cost monotonicity, DR is fundamentally leaky: fallback checks are pervasive (up to 40.6% of expansions), and eviction residues absorb up to 93% of expanded paths. Treating fallback as an incidental edge case leads to severe computational bottlenecks.

FAST-MVH resolves this breakdown by pairing adaptive truncated frontiers with a provably sufficient, logarithmically indexed residue forest, single-pass witness classification, and witness-cached heuristic pruning. It reproduces the exact search trajectory and Pareto fronts of L-NAMOA$^*_{dr}$-mvh while achieving an overall geometric mean speedup of 22.58x (and peak speedups up to 960.38x) across 4 to 8 objectives without a single timeout. By taming the fallback, FAST-MVH proves that geometric indexing of the eviction residue is the essential missing link that makes dimensionality reduction practical and scalable for multi-valued heuristic search.

References

Bentley, J. L. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM* 18(9): 509–517.

Bentley, J. L.; and Saxe, J. B. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1(4): 301–358.

Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series 74. American Mathematical Society.

Geisser, F.; Haslum, P.; Thiébaux, S.; and Trevizan, F. 2022. Admissible Heuristics for Multi-Objective Planning. In *Proceedings of ICAPS*, 100–109.

Pulido, F. J.; Mandow, L.; and Pérez-de-la-Cruz, J. L. 2015. Dimensionality Reduction in Multiobjective Shortest Path Search. *Computers & Operations Research* 64: 60–70.

Shperberg, S. S.; et al. 2026. A Geometric Index for Multi-Objective Dominance Checking. Manuscript under review.

Skyler, S.; Shperberg, S. S.; Atzmon, D.; Felner, A.; Salzman, O.; Chan, S.; Zhang, H.; Koenig, S.; Yeoh, W.; and Hernández Ulloa, C. 2024. Theoretical Study on Multi-Objective Heuristic Search. In *Proceedings of IJCAI*, 7021–7028.

Wolff, M.; Felner, A.; and Salzman, O. 2026. Bridging Multi-Valued Heuristics and Dimensionality Reduction in Multi-Objective Search. In *Proceedings of SoCS*.

Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernández Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A*pex: Efficient Approximate Multi-Objective Search on Graphs. In* Proceedings of ICAPS*, 394–403.

Appendix: Complete Benchmark Evaluation Matrix

Table A1 presents the exhaustive instance-by-instance evaluation matrix across all 28 canonical benchmark settings evaluated to natural completion without time limits. In 100% of the runs, FAST-MVH and its ablation variants produced cryptographically bit-identical Pareto sets to the reference baseline (verified via SHA-256 checksums). Sub-tenth-second runs are marked with † to denote instances where runtimes fall below the operating system timer resolution threshold.

Table A1. Complete Instance-by-Instance Evaluation Matrix across 28 Benchmark Settings ($M = 4 \ldots 8$). Runtimes in seconds evaluated to natural completion without time limits. Sub-tenth-second runs marked with † are filtered out from ratio geometric means to eliminate timer quantization noise. Every completed run returned 100% cryptographically bit-identical Pareto sets (SHA-256 verified).

Instance	Domain	M	ρ	Sols ($\lvert\Pi^*\rvert$)	Expansions	Baseline (s)	FAST-MVH	w/o Goal KDT	w/o Local KDT	w/o Fallback Forest	w/o Local-First	w/o Residue Filter	Bit-Identical
Panda_S242_G93	Robotics	8	0.0	13	130	0.01s	**0.004s** †	0.00s	0.00s	0.00s	0.00s	0.01s	Exact SHA-256 Match
Panda_S29_G93	Robotics	8	0.0	50	280	0.03s	**0.010s** †	0.01s	0.01s	0.02s	0.01s	0.01s	Exact SHA-256 Match
Panda_S29_G405	Robotics	8	0.0	46	6,711	0.60s	**0.22s**	0.16s	0.21s	0.15s	0.18s	0.19s	Exact SHA-256 Match
Panda_Q0024	Robotics	8	0.0	109	40,057	7.91s	**2.19s**	2.34s	2.31s	2.27s	2.71s	2.50s	Exact SHA-256 Match
Panda_Q0010	Robotics	8	0.0	2,203	93,945	244.03s	**68.94s**	69.86s	77.86s	67.81s	80.35s	70.06s	Exact SHA-256 Match
Panda_Q0002_Roi	Robotics	8	0.0	6,581	738,325	9124.07s (2.5h)	**34.61s**	98.90s	47.96s	32.80s	41.34s	31.59s	Exact SHA-256 Match
bay_8d_2000 (s500)	Roadmaps	5	0.0	5,529	36,915	8.59s	**0.42s**	9.72s	0.50s	0.41s	0.74s	0.41s	Exact SHA-256 Match
bay_8d_2000 (s2000)	Roadmaps	5	0.0	28,200	192,161	183.08s	**1.71s**	14.85s	3.36s	1.66s	2.41s	1.68s	Exact SHA-256 Match
grid10_M4_rho0.0	Grids	4	0.0	1,889	47,756	1.56s	**0.54s**	0.66s	0.61s	0.54s	0.88s	0.55s	Exact SHA-256 Match
grid10_M4_rho-0.2	Grids	4	-0.2	5,080	93,274	5.85s	**1.33s**	1.88s	1.62s	1.37s	1.90s	1.31s	Exact SHA-256 Match
grid10_M4_rho-0.4	Grids	4	-0.4	9,474	147,304	12.32s	**2.28s**	3.44s	2.93s	2.56s	3.43s	2.45s	Exact SHA-256 Match
grid12_M4_rho0.0	Grids	4	0.0	16,147	283,658	47.06s	**4.95s**	8.61s	7.45s	8.34s	8.30s	5.66s	Exact SHA-256 Match
grid15_M4_rho0.0	Grids	4	0.0	34,459	1,117,374	1268.39s	**18.09s**	21.15s	21.80s	17.30s	16.61s	13.20s	Exact SHA-256 Match
grid10_M5_rho0.0	Grids	5	0.0	23,150	269,107	144.53s	**6.96s**	23.46s	12.85s	8.27s	10.52s	7.39s	Exact SHA-256 Match
grid10_M5_rho-0.2	Grids	5	-0.2	48,303	475,255	818.97s	**32.38s**	48.07s	69.66s	40.51s	53.61s	34.56s	Exact SHA-256 Match
grid10_M5_rho-0.4	Grids	5	-0.4	58,746	546,830	2019.56s	**5.03s**	16.30s	13.47s	11.45s	15.67s	9.01s	Exact SHA-256 Match
grid12_M5_rho0.0	Grids	5	0.0	98,070	1,425,808	6048.91s (1.7h)	**137.72s**	489.32s	369.57s	291.95s	192.42s	128.79s	Exact SHA-256 Match
grid_6x6_M6_rho0.0	Grids	6	0.0	3,269	16,487	5.07s	**0.64s**	1.38s	0.85s	0.69s	1.95s	0.69s	Exact SHA-256 Match
grid_8x8_M6_rho0.0	Grids	6	0.0	71,405	295,885	3631.27s (1.0h)	**28.03s**	256.91s	110.46s	33.69s	47.86s	21.98s	Exact SHA-256 Match
grid_5x5_M7_rho0.0	Grids	7	0.0	1,694	5,370	0.94s	**0.16s**	0.25s	0.17s	0.16s	0.33s	0.17s	Exact SHA-256 Match
grid_6x6_M7_rho0.0	Grids	7	0.0	8,987	38,515	19.96s	**2.67s**	11.28s	5.51s	2.58s	6.48s	4.06s	Exact SHA-256 Match
grid_7x7_M7_rho0.0	Grids	7	0.0	59,157	221,389	2934.48s	**23.14s**	187.87s	83.81s	32.36s	43.27s	29.01s	Exact SHA-256 Match
grid_8x8_M7_rho0.0	Grids	7	0.0	192,162	823,682	27666.05s (7.7h)	**191.88s**	3122.31s	1340.46s	217.17s	277.22s	186.05s	Exact SHA-256 Match
grid_5x5_M8_rho0.0	Grids	8	0.0	5,877	13,819	12.77s	**0.78s**	2.97s	1.10s	0.76s	1.28s	0.77s	Exact SHA-256 Match
grid_6x6_M8_rho0.0	Grids	8	0.0	22,726	65,839	221.11s	**6.53s**	41.98s	13.86s	5.69s	10.30s	6.27s	Exact SHA-256 Match
grid_6x6_M8_rho-0.2	Grids	8	-0.2	28,284	83,866	321.82s	**10.88s**	43.37s	16.47s	8.56s	11.83s	8.13s	Exact SHA-256 Match
grid_7x7_M8_rho0.0	Grids	8	0.0	75,169	287,802	5491.50s (1.5h)	**49.57s**	500.33s	200.54s	49.73s	68.88s	45.96s	Exact SHA-256 Match
grid_8x8_M8_rho0.0	Grids	8	0.0	369,351	1,521,453	89469.10s (24.9h)	**93.16s**	2267.75s	724.04s	95.53s	155.44s	94.17s	Exact SHA-256 Match